## 🧠 Concept Overview: What is a Graph?

A **Graph** is a **non-linear data structure** that represents a set of **objects (called vertices or nodes)** connected by **links (called edges)**.

In simple terms:

A Graph = **Nodes (Vertices)** + **Connections (Edges)**

For example:

A ---- B

|  |

|  |

C ---- D

Here:

- **Vertices (Nodes)** = {A, B, C, D}
- **Edges (Connections)** = {(A,B), (A,C), (B,D), (C,D)}

---

### 🔹 Types of Graphs

| Type | Description | Example |
|---|---|---|
| **Undirected Graph** | Edges have **no direction**. Connection is mutual. | (A, B) means both A→B and B→A |
| **Directed Graph (Digraph)** | Edges have **direction**. | A → B (but not necessarily B → A) |
| **Weighted Graph** | Each edge has a **weight/cost** (like distance or time). | A → B (cost = 5) |
| **Unweighted Graph** | Edges have **no weights**. | A → B |
| **Cyclic Graph** | Contains at least one cycle (a closed path). | A → B → C → A |
| **Acyclic Graph** | No cycles. | Tree structure |

| Type | Description | Example |
|---|---|---|
| **Connected Graph** | All vertices are reachable from any other vertex. | Fully linked graph |
| **Disconnected Graph** | Some vertices are isolated. | A ↔ B, but C has no connection |

---

### ⚙️ Graph Representation in Python

There are two main ways to represent graphs:

#### 1️⃣ Adjacency Matrix

A 2D array (matrix) where:

- Rows and columns represent vertices
- A 1 (or weight) means there's an edge between them

Example:

Vertices: A, B, C

```
   A B C
A [ 0 1 0 ]
B [ 1 0 1 ]
C [ 0 1 0 ]
```

Python Example:

```
# Adjacency Matrix Representation
graph = [
   [0, 1, 0],
   [1, 0, 1],
   [0, 1, 0]
]
```

---

### 2️⃣ Adjacency List

Each vertex stores a list of connected vertices.

Example:

A → [B]

B → [A, C]

C → [B]

Python Example:

```
# Adjacency List Representation using Dictionary
graph = {
    'A': ['B'],
    'B': ['A', 'C'],
    'C': ['B']
}
```

---

### 🔄 Graph Traversal

Traversal means visiting all vertices in a systematic order.

#### 1️⃣ Breadth-First Search (BFS)

- Uses **Queue (FIFO)**
- Visits nodes level by level

```
from collections import deque


def bfs(graph, start):
    visited = set()
    queue = deque([start])
```

```
    while queue:
        node = queue.popleft()
        if node not in visited:
            print(node, end=" ")
            visited.add(node)
            queue.extend(neighbor for neighbor in graph[node] if neighbor not in visited)

graph = {
    'A': ['B', 'C'],
    'B': ['A', 'D'],
    'C': ['A', 'D'],
    'D': ['B', 'C']
}
bfs(graph, 'A')
# Output: A B C D
```

---

### 2️⃣ Depth-First Search (DFS)

- Uses **Stack (LIFO)** or recursion
- Explores as far as possible along each branch before backtracking

```
def dfs(graph, node, visited=None):
    if visited is None:
        visited = set()

    print(node, end=" ")
    visited.add(node)
```

```
    for neighbor in graph[node]:
        if neighbor not in visited:
            dfs(graph, neighbor, visited)

graph = {
    'A': ['B', 'C'],
    'B': ['D'],
    'C': ['D'],
    'D': []
}
dfs(graph, 'A')
# Output: A B D C
```

---

### 💡 Use Cases of Graphs

| Domain | Example | Description |
|---|---|---|
| **Social Networks** | Facebook, LinkedIn | Users = vertices, Friendships = edges |
| **Maps & Navigation** | Google Maps | Cities = vertices, Roads = weighted edges |
| **Recommendation Systems** | Netflix, Amazon | Graphs link users to items or genres |
| **Computer Networks** | Internet topology | Routers = nodes, Connections = edges |
| **Web Crawlers** | Google Search | Web pages connected via hyperlinks |
| **Scheduling Problems** | Project Management | Tasks and dependencies form a graph |
| **AI & Machine Learning** | Graph Neural Networks (GNN) | Used for relationship modeling |

---

## 📘 Summary Table

| Concept | Description |
|---|---|
| **Graph** | Collection of vertices and edges |
| **Vertex (Node)** | Represents an entity |
| **Edge** | Represents a relationship |
| **Directed / Undirected** | One-way or two-way connections |
| **Weighted / Unweighted** | Edges may carry weights |
| **Traversal** | BFS (level-wise), DFS (depth-wise) |
| **Storage** | Adjacency List or Matrix |